

Part01 – Questions & Answers

Q1: Why can't a struct inherit from another struct or class in C#?

A: In C#, every struct implicitly derives from `System.ValueType` (which itself derives from `System.Object`), and the language does not allow it to derive from any other type, nor can another struct or class derive from it — structs are implicitly sealed with respect to inheritance (though they can still implement interfaces). This is a deliberate design choice: structs are meant to be small, lightweight, stack-friendly value types that are copied by value. Supporting inheritance would require the same mechanisms classes use for polymorphism (a type identity/vtable, reference semantics for correct overriding, and boxing to treat a derived struct as its base), which would defeat the whole purpose of a struct being a fast, self-contained block of data. So the language restricts structs to implementing interfaces only, and keeps full inheritance exclusive to classes, which are reference types designed for that purpose.

Q2: How do access modifiers impact the scope and visibility of a class member?

A: Access modifiers control which parts of the code are allowed to see and use a class member (field, property, method, etc.). `private` restricts access to only the containing type itself. `internal` allows access from anywhere within the same assembly (project), but not from other assemblies. `public` allows access from anywhere, including other assemblies/projects. `protected` allows access from the containing class and any class that inherits from it. `protected internal` combines `protected` and `internal` (accessible within the same assembly OR from a derived class in another assembly). `private protected` is the most restrictive combination — accessible only within the same assembly AND only from a derived class. Choosing the right modifier directly shapes the public 'contract' of a class: it determines what other code can rely on, and it protects internal implementation details from being accessed or changed in unintended ways from outside.

Q3: Why is encapsulation critical in software design?

A: Encapsulation means bundling an object's data together with the methods that operate on that data, while hiding the internal state from the outside world (usually via private fields exposed through public properties or methods). It is critical because: (1) it protects data integrity, since the object controls how its own state can be changed instead of letting any outside code set it to an invalid value directly; (2) it reduces coupling between different parts of a program, because outside code depends only on a stable public interface, not on internal implementation details; (3) it makes the code easier to maintain and evolve, since the internal implementation can be changed freely as long as the public interface stays the same; (4) it improves security and reliability, since validation logic (e.g. inside a setter) can prevent invalid states. Without encapsulation, any part of a program could freely modify another object's internal data, making bugs far more likely and the codebase much harder to reason about as it grows.

Q4: what is constructors in structs?

A: A constructor in a struct is a special method that runs when an instance of the struct is created, and its job is to initialize the struct's fields. Structs always have an implicit parameterless constructor supplied by the compiler, which sets every field to its default value (0, false, null, etc.) — this exists even if you never write any constructor yourself. You can also define your own parameterized constructors, exactly like in a class, to initialize the fields with specific values passed in by the caller; a struct can have multiple parameterized constructors (constructor overloading). Historically (before C# 10.0 / .NET 6.0), structs were not allowed to define their own explicit parameterless constructor — only the compiler-provided one was allowed — because the runtime needed a guaranteed, simple, zero-initialization path for structs (used, for example, when creating an array of structs or calling `default(T)`). Starting with C# 10.0, this restriction was relaxed, and structs are now allowed to define an explicit parameterless constructor with custom initialization logic as well.

Q5: How does overriding methods like ToString() improve code readability?

A: By default, `ToString()` (inherited from `System.Object`) returns the full type name of the object (e.g. `'CSharpTask06.Point'`), which is not useful for understanding the object's actual state. Overriding `ToString()` lets you provide a custom, meaningful string representation of the object (for example, printing a `Point` as `'(3, 5)'` instead of its type name). This directly improves readability because: printing an object with `Console.WriteLine` or including it

in a log message immediately shows useful information; it makes debugging faster, since you can see an object's actual data at a glance instead of its type name; and it keeps calling code clean, since callers don't need to manually format each field themselves every time they want to display the object.

Q6: How does memory allocation differ for structs and classes in C#?

A: Structs are value types: an instance's data is generally stored directly wherever the variable lives — on the stack for a local variable, or inline inside the containing object if the struct is a field of a class or an element of an array. When a struct is assigned to another variable or passed to a method, its entire content is copied, producing an independent copy. Classes are reference types: the actual object data is allocated on the managed heap, and the variable itself only holds a reference (a pointer) to that heap location. When a class instance is assigned to another variable or passed to a method, only the reference is copied — both variables end up pointing to the same object in memory, so changes made through one variable are visible through the other. This difference is also why structs are generally cheaper for small, short-lived data (no heap allocation, no garbage collection pressure), while classes are more suitable for larger objects or objects that need shared, mutable identity across different parts of a program.

Part02 – Questions & Answers

Q1: What is copy constructor?

A: A copy constructor is a constructor whose purpose is to create a new object by copying the field values of an existing object of the same type. C# does not generate one automatically (unlike C++); if you want one, you define it yourself as a constructor that takes a single parameter of the same type and copies its fields into the new instance, for example: `public Employee(Employee other) { this.name = other.name; this.salary = other.salary; }`. Copy constructors are especially useful for classes, since a normal assignment (`Employee e2 = e1;`) only copies the reference, leaving both variables pointing to the same object — a copy constructor lets you create a genuinely separate object with the same data. For structs this concern doesn't really apply, since a simple assignment already copies all the field values automatically due to value-type semantics.

Q2: What is Indexer, when used, as business mention cases u have to utilize it?

A: An indexer is a special class member that allows instances of a class or struct to be accessed using array-like syntax (`object[index]`) instead of calling a named method. It's defined using the 'this' keyword with get/set accessors, for example: `public string this[int index] { get { ...; } set { ...; } }`. Indexers are used when a class logically represents a collection, sequence, or lookup of related items, and you want a natural, array-like way to read or write individual elements without exposing the internal storage (array, list, dictionary) directly. Typical business use cases include: a ShoppingCart class where items are accessed by product code (`cart["SKU123"]`); a Configuration/Settings class where values are read by key (`config["ConnectionTimeout"]`); a Matrix or Grid class in a scientific or financial application, accessed as `matrix[row, col]`; or a custom read-only collection class in a business layer that also performs validation, logging, or lazy-loading each time an item is accessed by index, something a plain public array field could not do.

Q3: Summarize keywords we have learnt last lecture

A: Based on the topics covered across this lecture's problems and self-study material, the key keywords and concepts are: struct and class (value type vs. reference type), constructors (default/implicit constructor, parameterized constructors, constructor overloading, and the C# 10.0/.NET 6.0 change allowing explicit parameterless constructors in structs), the override keyword and overriding ToString() for custom, readable object representation, encapsulation (private fields exposed through public methods or properties), access modifiers (private, internal, public, protected, protected internal, private protected) and how each controls visibility scope, properties (get/set accessors) as a controlled way to expose private fields, the difference in memory allocation and copying behavior between structs (stack/inline, copied by value) and classes (heap, copied by reference), the copy constructor as a way to create an independent copy of a reference-type object, and the indexer as a way to expose array-like, index-based access to a class's internal data.

Part03 – Self Study Topic

Struct Parameterless Constructors: Default Initialization and the C# 10.0 / .NET 6.0 Change

Every struct in C# has always had an implicit, compiler-generated parameterless constructor. Its role is simply to initialize every field of the struct to its default value the moment an instance is created with 'new StructName()' — numeric fields become 0, boolean fields become false, and reference-type fields become null. This guarantees that a struct instance is never left in an undefined or garbage state; it is always fully and predictably zero-initialized, even if the developer never wrote any constructor at all.

Before C# 10.0 (which shipped together with .NET 6.0), the language deliberately did not allow a struct to declare its own explicit parameterless constructor. Writing 'public Point() { ... }' inside a struct produced compiler error CS0568 ('Structs cannot contain explicit parameterless constructors'). The reason behind this restriction was consistency and predictability: the CLR can create struct instances through several different paths that do not go through any constructor at all — for example, creating an array of structs (new Point[10]), using the default(T) expression, or having a struct as an uninitialized field of another type. In all of these cases, the runtime simply zero-fills the memory rather than calling a constructor. If developers had been allowed to write their own custom parameterless constructor, it would only run for the explicit 'new Point()' case and would be silently skipped in the other cases, creating a dangerous inconsistency where the exact same struct could end up either fully custom-initialized or plain zero-initialized depending on how it happened to be created — a subtle and hard-to-debug source of bugs. To avoid this, the language simply forbade custom parameterless constructors entirely, keeping the guarantee simple: 'no explicit constructor = always zero-initialized, no exceptions.'

Starting with C# 10.0 / .NET 6.0, this restriction was relaxed: structs are now allowed to define their own explicit parameterless constructor containing custom initialization logic (for example, setting a field to a non-zero default value instead of 0). This became useful for struct designs where a 'sensible default' is not simply zero — for instance, a struct representing a percentage might want to default to 100 instead of 0. However, the underlying inconsistency described above did not disappear; it was simply made an explicit, documented trade-off that the developer opts into. The explicit parameterless constructor only executes when the struct is created via 'new StructName()' in code. It is still bypassed in scenarios such as creating an array of the struct (new StructName[5]) or using default(StructName) — those cases still produce a plain, zero-initialized instance, without running the custom constructor. So the practical rule to remember is: the role of a (implicit or explicit) parameterless constructor is to guarantee that every field starts out in a valid, known state; but a developer relying on an explicit parameterless constructor in a struct must be aware that not every creation path in C# actually invokes it, unlike a class, where the constructor is always guaranteed to run for every 'new' instantiation.